Django Complete Notes

1. URL Dispatcher

What is URL Dispatcher?

Django URL Dispatcher maps a requested URL to the appropriate view function or class.

Example

```
# urls.py

from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
    path("about/", views.about, name="about"),
    path("products/<int:product_id>/", views.product_detail,
name="product_detail"),
]
```

View Functions

```
# views.py

from django.http import HttpResponse

def home(request):
    return HttpResponse("Home Page")

def about(request):
    return HttpResponse("About Page")

def product_detail(request, product_id):
    return HttpResponse(f"Product ID: {product_id}")
```

URL Parameters

```
path("post/<int:id>/", views.post_detail)
path("user/<str:username>/", views.profile)
path("article/<slug:slug>/", views.article_detail)
path("file/<path:file_path>/", views.file_view)
```

Common path converters:

- `str`: Matches a non-empty string excluding `/`.
- `int`: Matches a positive integer.
- `slug`: Matches letters, numbers, hyphens, and underscores.
- `uuid`: Matches a UUID.
- `path`: Matches a non-empty path, including `/`.

include()

Use `include()` to organize URLs by application.

```
# project/urls.py

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("blog/", include("blog.urls")),
]
```

```
# blog/urls.py

from django.urls import path
from . import views

app_name = "blog"

urlpatterns = [
    path("", views.post_list, name="post_list"),
    path("<int:pk>/", views.post_detail, name="post_detail"),
]
```

Reverse URL Resolution

```
from django.urls import reverse

url = reverse("blog:post_detail", args=[5])
```

In templates:

```
<a href="{% url 'blog:post_list' %}">All Posts</a>
<a href="{% url 'blog:post_detail' pk=post.pk %}">Read More</a>
```

Important: Use named URLs instead of hardcoding URLs whenever possible.

---

2. Writing Views

What is a View?

A view is a Python function or class that receives an HTTP request and returns an HTTP response.

Function-Based View (FBV)

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello Django!")
```

Rendering an HTML Template

```
from django.shortcuts import render

def home(request):
    context = {
        "name": "Faruk",
        "language": "Python",
    }

    return render(request, "home.html", context)
```

Template:

```
<h1>Hello, {{ name }}</h1>
<p>Language: {{ language }}</p>
```

Handling GET and POST Requests

```
from django.shortcuts import render
from django.http import HttpResponse

def contact(request):
    if request.method == "POST":
```

```
            name = request.POST.get("name", "")
            return HttpResponse(f"Hello, {name}")

        return render(request, "contact.html")
```

Template:

```
<form method="post">
    {% csrf_token %}

    <input type="text" name="name" placeholder="Your name">

    <button type="submit">Submit</button>
</form>
```

Returning JSON

```
from django.http import JsonResponse

def api_view(request):
    data = {
        "name": "Django",
        "version": "6.0",
        "success": True,
    }

    return JsonResponse(data)
```

Raising a 404 Error

```
from django.shortcuts import get_object_or_404
from .models import Post

def post_detail(request, pk):
    post = get_object_or_404(Post, pk=pk)

    return render(request, "post_detail.html", {
        "post": post,
    })
```

Class-Based View (CBV)

```
from django.views import View
from django.http import HttpResponse

class HomeView(View):
    def get(self, request):
        return HttpResponse("GET request")

    def post(self, request):
        return HttpResponse("POST request")
```

URL:

```
path("", HomeView.as_view(), name="home")
```

---

3. View Decorators

Decorators modify or restrict the behavior of views.

require_GET

Allows only GET requests.

```
from django.views.decorators.http import require_GET

@require_GET
def home(request):
    return HttpResponse("GET only")
```

require_POST

Allows only POST requests.

```
from django.views.decorators.http import require_POST
from django.http import HttpResponse

@require_POST
def submit_form(request):
    return HttpResponse("Form submitted")
```

require_safe

Allows GET and HEAD requests.

```
from django.views.decorators.http import require_safe
```

```
@require_safe
def article_list(request):
    return HttpResponse("Articles")
```

require_http_methods

```
from django.views.decorators.http import require_http_methods

@require_http_methods(["GET", "POST"])
def my_view(request):
    if request.method == "POST":
        return HttpResponse("POST request")

    return HttpResponse("GET request")
```

login_required

Requires the user to be logged in.

```
from django.contrib.auth.decorators import login_required
from django.http import HttpResponse

@login_required
def dashboard(request):
    return HttpResponse("Dashboard")
```

permission_required

Requires a specific permission.

```
from django.contrib.auth.decorators import permission_required

@permission_required("blog.add_post", raise_exception=True)
def create_post(request):
    return HttpResponse("Create Post")
```

csrf_protect

Enforces CSRF protection for a view.

```
from django.views.decorators.csrf import csrf_protect

@csrf_protect
```

```
    def my_form(request):
        return render(request, "form.html")
```

Use `{% csrf_token %}` inside internal POST forms.

cache_page

Caches the response for a specified number of seconds.

```
from django.views.decorators.cache import cache_page

@cache_page(60 * 15)
def article_list(request):
    return render(request, "articles.html")
```

This example configures a 15-minute cache.

Decorators for Class-Based Views

```
from django.contrib.auth.decorators import login_required
from django.utils.decorators import method_decorator
from django.views import View

@method_decorator(login_required, name="dispatch")
class DashboardView(View):
    def get(self, request):
        return HttpResponse("Dashboard")
```

---

4. File Uploads

Configure Media Settings

```
# settings.py

MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Model with FileField and ImageField

```
# models.py

from django.db import models

class Post(models.Model):
```

```
    title = models.CharField(max_length=200)
    document = models.FileField(
        upload_to="documents/",
        blank=True,
    )
    image = models.ImageField(
        upload_to="images/",
        blank=True,
    )
```

Install Pillow for `ImageField`:

```
pip install Pillow
```

Create a Model Form

```
# forms.py

from django import forms
from .models import Post

class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["title", "document", "image"]
```

Handle File Uploads

```
# views.py

from django.shortcuts import render, redirect
from .forms import PostForm

def upload_post(request):
    if request.method == "POST":
        form = PostForm(
            request.POST,
            request.FILES,
        )

        if form.is_valid():
            form.save()
```

```
                return redirect("upload_post")

        else:
            form = PostForm()

        return render(request, "upload.html", {
            "form": form,
        })
```

Upload Template

```
<h1>Upload Post</h1>

<form method="post" enctype="multipart/form-data">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Upload</button>
</form>
```

Important: File upload forms require `enctype="multipart/form-data"` and the view must receive `request.FILES`.

Display an Uploaded Image

```
{% if post.image %}
    <img src="{{ post.image.url }}" alt="{{ post.title }}">
{% endif %}
```

Serve Media During Development

```
# project/urls.py

from django.conf import settings
from django.conf.urls.static import static
from django.urls import path, include

urlpatterns = [
    path("blog/", include("blog.urls")),
]
```

```
if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT,
    )
```

This development setup is not a production media-serving solution.

---

5. Django Shortcut Functions

render()

Renders a template and returns an HTTP response.

```
from django.shortcuts import render

def home(request):
    context = {"name": "Django"}

    return render(request, "home.html", context)
```

redirect()

Redirects to a URL, view name, or model object's `get_absolute_url()`.

```
from django.shortcuts import redirect

def home(request):
    return redirect("blog:post_list")
```

Redirect to an external URL:

```
return redirect("https://www.djangoproject.com/")
```

get_object_or_404()

Returns an object if it exists; otherwise raises `Http404`.

```
from django.shortcuts import get_object_or_404
from .models import Post

def post_detail(request, pk):
    post = get_object_or_404(Post, pk=pk)

    return render(request, "post_detail.html", {
```

```
            "post": post,
        })
```

get_list_or_404()

Returns a non-empty list of objects or raises `Http404`.

```
from django.shortcuts import get_list_or_404
from .models import Post

def post_list(request):
    posts = get_list_or_404(Post, is_published=True)

    return render(request, "post_list.html", {
        "posts": posts,
    })
```

The `is_published` field must exist on the model.

---

6. Generic Views

Generic class-based views reduce repetitive code.

TemplateView

Displays a template.

```
from django.views.generic import TemplateView

class HomeView(TemplateView):
    template_name = "home.html"
```

URL:

```
path("", HomeView.as_view(), name="home")
```

ListView

Displays a list of objects.

```
from django.views.generic import ListView
from .models import Post

class PostListView(ListView):
    model = Post
    template_name = "post_list.html"
```

```
    context_object_name = "posts"
    ordering = ["-created_at"]
```

Template:

```
{% for post in posts %}
    <h2>{{ post.title }}</h2>
{% empty %}
    <p>No posts found.</p>
{% endfor %}
```

DetailView

Displays one object.

```
from django.views.generic import DetailView
from .models import Post

class PostDetailView(DetailView):
    model = Post
    template_name = "post_detail.html"
    context_object_name = "post"
```

CreateView

Creates an object.

```
from django.urls import reverse_lazy
from django.views.generic import CreateView
from .models import Post

class PostCreateView(CreateView):
    model = Post
    fields = ["title", "content"]
    template_name = "post_form.html"
    success_url = reverse_lazy("blog:post_list")
```

UpdateView

Updates an existing object.

```
from django.views.generic import UpdateView
from .models import Post

class PostUpdateView(UpdateView):
```

```
    model = Post
    fields = ["title", "content"]
    template_name = "post_form.html"
    success_url = reverse_lazy("blog:post_list")
```

DeleteView

Deletes an object after confirmation through a POST request.

```
from django.views.generic import DeleteView
from django.urls import reverse_lazy
from .models import Post

class PostDeleteView(DeleteView):
    model = Post
    template_name = "post_confirm_delete.html"
    success_url = reverse_lazy("blog:post_list")
```

Delete template:

```
<h1>Delete Post</h1>

<p>Are you sure you want to delete "{{ object }}"?</p>

<form method="post">
    {% csrf_token %}
    <button type="submit">Confirm Delete</button>
</form>
```

Generic View URLs

```
from django.urls import path
from .views import (
    PostListView,
    PostDetailView,
    PostCreateView,
    PostUpdateView,
    PostDeleteView,
)

app_name = "blog"

urlpatterns = [
```

```
    path("", PostListView.as_view(), name="post_list"),
    path("<int:pk>/", PostDetailView.as_view(), name="post_detail"),
    path("create/", PostCreateView.as_view(), name="post_create"),
    path("<int:pk>/edit/", PostUpdateView.as_view(),
name="post_update"),
    path("<int:pk>/delete/", PostDeleteView.as_view(),
name="post_delete"),
]
```

Security note: Protect create, update, and delete views with authentication and appropriate permissions. If users should manage only their own posts, enforce ownership checks in the view.

---

7. Middleware

What is Middleware?

Middleware is a framework-level component that processes requests and responses globally.

Typical flow:

```
Client Request
     |
     v
Middleware (Request)
     |
     v
URL Resolver
     |
     v
View
     |
     v
Middleware (Response)
     |
     v
Client Response
```

Configure Middleware

```
# settings.py
```

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

Keep the required middleware and ordering appropriate for your project.

Create Custom Middleware

```
# middleware.py

class SimpleMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Process request before the view
        print("Before view")

        response = self.get_response(request)

        # Process response after the view
        print("After view")

        return response
```

Register it:

```
MIDDLEWARE = [
    # Other middleware...
    "myapp.middleware.SimpleMiddleware",
]
```

Replace `myapp` with your actual app package.

Middleware with process_view()

```
from django.http import HttpResponse

class ExampleMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        return self.get_response(request)

    def process_view(self, request, view_func, view_args, view_kwargs):
        print("View function:", view_func.__name__)
        return None
```

Returning `None` allows normal view processing. Returning an `HttpResponse` short-circuits the request and prevents the view from running.

Middleware vs Decorators

- Middleware can apply behavior across many views or the whole project.
- Decorators apply behavior to specific views or functions.
- Middleware runs as part of Django's request/response processing chain.
- Decorators wrap a view callable.

---

8. Django Sessions

What is a Session?

A session allows Django to associate data with a visitor across multiple requests.

By default, Django stores session data on the server and uses a cookie to identify the session.

Enable Sessions

Check `settings.py` for:

```
INSTALLED_APPS = [
    "django.contrib.sessions",
]

MIDDLEWARE = [
    "django.contrib.sessions.middleware.SessionMiddleware",
]
```

Run migrations when using the database-backed session engine:

```
python manage.py migrate
```

Set and Get Session Data

```
def set_session(request):
    request.session["username"] = "Faruk"
    return HttpResponse("Session saved")
```

```
def get_session(request):
    username = request.session.get("username", "Guest")
    return HttpResponse(f"Hello, {username}")
```

Update and Delete Session Data

```
request.session["theme"] = "dark"
```

```
request.session.pop("theme", None)
```

Clear the Session

```
request.session.flush()
```

`flush()` removes the current session data and session cookie and resets the session key.

Session Expiry

```
request.session.set_expiry(300)
```

This sets the session to expire after 300 seconds of inactivity.

Other options:

```
request.session.set_expiry(0)
```

Expires when the browser is closed, subject to browser behavior.

```
request.session.set_expiry(None)
```

Uses the configured session expiry policy.

Session Counter

```
from django.http import HttpResponse

def counter(request):
    count = request.session.get("count", 0)
    count += 1

    request.session["count"] = count

    return HttpResponse(f"Visited {count} times")
```

Session Settings

```
SESSION_COOKIE_AGE = 1209600
SESSION_SAVE_EVERY_REQUEST = False
SESSION_EXPIRE_AT_BROWSER_CLOSE = False
```

These are examples of common settings; use values that match your application's requirements.

Sessions vs Cookies

- A cookie is stored in the browser.
- A session is a server-side mechanism for maintaining visitor-specific state, commonly identified by a cookie.
- Do not store passwords or other secrets directly in session data.
- Session data should not be trusted as authorization by itself; enforce permissions on the server.

---

9. Practice Project: Django Blog

Step 1: Create a Project and App

```
django-admin startproject config
cd config
python manage.py startapp blog
```

Add the app in `settings.py`:

```
INSTALLED_APPS = [
    # Default Django apps...
    "blog",
]
```

Step 2: Create a Model

```
# blog/models.py

from django.db import models
from django.urls import reverse

class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return self.title

    def get_absolute_url(self):
        return reverse("blog:post_detail", kwargs={"pk": self.pk})
```

Step 3: Create Forms

```
# blog/forms.py

from django import forms
from .models import Post

class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["title", "content"]
```

Step 4: Create Views

```
# blog/views.py

from django.contrib.auth.decorators import login_required
from django.shortcuts import render
from django.urls import reverse_lazy
from django.utils.decorators import method_decorator
from django.views.generic import (
```

```
    ListView,
    DetailView,
    CreateView,
    UpdateView,
    DeleteView,
)
from .models import Post

class PostListView(ListView):
    model = Post
    template_name = "blog/post_list.html"
    context_object_name = "posts"

class PostDetailView(DetailView):
    model = Post
    template_name = "blog/post_detail.html"
    context_object_name = "post"

@method_decorator(login_required, name="dispatch")
class PostCreateView(CreateView):
    model = Post
    fields = ["title", "content"]
    template_name = "blog/post_form.html"

@method_decorator(login_required, name="dispatch")
class PostUpdateView(UpdateView):
    model = Post
    fields = ["title", "content"]
    template_name = "blog/post_form.html"

@method_decorator(login_required, name="dispatch")
class PostDeleteView(DeleteView):
    model = Post
    template_name = "blog/post_confirm_delete.html"
    success_url = reverse_lazy("blog:post_list")
```

Authentication alone does not restrict users to their own posts. Add ownership and permission checks where required.

Step 5: Configure URLs

```
# blog/urls.py

from django.urls import path
from .views import (
    PostListView,
    PostDetailView,
    PostCreateView,
    PostUpdateView,
    PostDeleteView,
)

app_name = "blog"

urlpatterns = [
    path("", PostListView.as_view(), name="post_list"),
    path("<int:pk>/", PostDetailView.as_view(), name="post_detail"),
    path("create/", PostCreateView.as_view(), name="post_create"),
    path("<int:pk>/edit/", PostUpdateView.as_view(),
name="post_update"),
    path("<int:pk>/delete/", PostDeleteView.as_view(),
name="post_delete"),
]
```

Project URLs:

```
# config/urls.py

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("blog/", include("blog.urls")),
]
```

Step 6: Create Templates

Create this directory structure:

```
blog/
    templates/
        blog/
```

```
                post_list.html
                post_detail.html
                post_form.html
                post_confirm_delete.html
```

`post_list.html`:

```
<h1>Blog Posts</h1>

<a href="{% url 'blog:post_create' %}">Create Post</a>

{% for post in posts %}
    <article>
        <h2>
            <a href="{{ post.get_absolute_url }}">
                {{ post.title }}
            </a>
        </h2>

        <p>{{ post.created_at }}</p>
        <p>{{ post.content|truncatewords:30 }}</p>
    </article>
{% empty %}
    <p>No posts available.</p>
{% endfor %}
```

`post_detail.html`:

```
<h1>{{ post.title }}</h1>

<p>{{ post.created_at }}</p>

<div>{{ post.content|linebreaks }}</div>

<a href="{% url 'blog:post_update' pk=post.pk %}">Edit</a>
<a href="{% url 'blog:post_delete' pk=post.pk %}">Delete</a>
<a href="{% url 'blog:post_list' %}">Back</a>
```

`post_form.html`:

```
<h1>Post Form</h1>
```

```
    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Save</button>
    </form>
```

`post_confirm_delete.html`:

```
    <h1>Delete Post</h1>

    <p>Delete "{{ object.title }}"?</p>

    <form method="post">
        {% csrf_token %}

        <button type="submit">Confirm Delete</button>
    </form>
```

Step 7: Run the Project

```
    python manage.py makemigrations
    python manage.py migrate
    python manage.py createsuperuser
    python manage.py runserver
```

Open:

```
    http://127.0.0.1:8000/blog/
```

---

10. Django Interview Questions and Answers

1. What is URL Dispatcher?

It maps URL patterns to the appropriate view.

2. What is URLconf?

A Python module containing URL patterns for an application or project.

3. What is the difference between FBV and CBV?

FBVs use functions; CBVs use classes and can support inheritance and reusable behavior.

4. What does render() do?

It renders a template with a context and returns an HTTP response.

5. What does redirect() do?

It returns a redirect response to another URL.

6. What does get_object_or_404() do?

It returns an object or raises `Http404` if the object is not found.

7. What is a decorator?

A callable that wraps another callable to modify or extend its behavior.

8. What is login_required?

A decorator that requires a user to be authenticated before accessing a view.

9. What is CSRF protection?

Protection against certain forged requests made using a user's authenticated browser session.

10. What is request.FILES?

A request attribute containing uploaded files for an appropriate multipart form submission.

11. What is MEDIA_ROOT?

The filesystem directory where uploaded media files are stored.

12. What is MEDIA_URL?

The URL prefix used to construct URLs for media files.

13. What is ListView?

A generic class-based view for displaying a list of objects.

14. What is DetailView?

A generic class-based view for displaying a single object.

15. What is CreateView?

A generic class-based view for creating an object with a form.

16. What is UpdateView?

A generic class-based view for updating an object.

17. What is DeleteView?

A generic class-based view for deleting an object, normally after a confirmation step.

18. What is middleware?

A component in Django's request/response processing chain.

19. What is a session?

A mechanism for maintaining visitor-specific data across requests.

20. What is the difference between session and cookie?

A cookie is stored in the browser; Django sessions commonly store data server-side and use a cookie to identify the session.

21. What is include()?

It includes URL patterns from another URL configuration module.

22. What is reverse()?

It resolves a named URL pattern into a URL string.

23. What is reverse_lazy()?

It delays URL resolution until the URL is needed, which is useful in class attributes.

24. Why use named URLs?

They make URL references easier to maintain when URL patterns change.

25. Why is authentication not enough for object-level security?

A logged-in user may still be unauthorized to edit or delete another user's object. Enforce ownership and permissions explicitly.

---

11. Learning Checklist

- Understand `path()`, `include()`, and URL converters.
- Create function-based and class-based views.
- Handle GET and POST requests.
- Render templates and return JSON.
- Use decorators for request methods and authentication.
- Upload and display files.
- Use `render()`, `redirect()`, and `get_object_or_404()`.
- Implement ListView and DetailView.
- Implement CreateView, UpdateView, and DeleteView.
- Create and register custom middleware.
- Set, retrieve, and clear session data.
- Build a CRUD application.

- Apply authentication, authorization, CSRF protection, and upload validation.

12. Official Django Documentation

1. URL Dispatcher: https://docs.djangoproject.com/en/6.0/topics/http/urls/
2. Writing Views: https://docs.djangoproject.com/en/6.0/topics/http/views/
3. View Decorators: https://docs.djangoproject.com/en/6.0/topics/http/decorators/
4. File Uploads: https://docs.djangoproject.com/en/6.0/topics/http/file-uploads/
5. Shortcut Functions: https://docs.djangoproject.com/en/6.0/topics/http/shortcuts/
6. Generic Views: https://docs.djangoproject.com/en/6.0/topics/class-based-views/generic/
7. Middleware: https://docs.djangoproject.com/en/6.0/topics/http/middleware/
8. Sessions: https://docs.djangoproject.com/en/6.0/topics/http/sessions/